# Нотация IDEF0

Инструментальные средства разработки · 3 курс · 4 часа (2 лекции по 2 часа)

## Лекция 1

Теория, история, базовые принципы и компоненты нотации IDEF0

## Лекция 2

Декомпозиция, правила построения, практический пример и инструменты

Цель курса: научить студентов применять функциональное моделирование как инструмент структурного анализа перед проектированием информационных систем.

# От проблемы к коду: зачем программисту моделировать бизнес-процессы

## IDEFo как инструмент функционального проектирования

Стандарт **IEEE 1320.1–1998** определяет IDEF0 как язык функционального моделирования, позволяющий описывать бизнес-процессы в виде иерархии функций с явными потоками данных и ресурсов.

IDEF0 — не просто диаграмма. Это формальный контракт между аналитиком и разработчиком о том, *что* система должна делать, прежде чем решается вопрос *как*.

## Связь с CASE-средствами

Структурный анализ на основе IDEF0 является фундаментом для проектирования ИС в CASE-инструментах: модели экспортируются в ERwin, BPwin, Rational Rose и другие среды.

## Практическая ценность

Модель IDEF0 позволяет понять требования к системе **до написания кода**: выявить входные данные, управляющие воздействия, исполнителей и ожидаемые результаты каждой функции. Это снижает стоимость исправления ошибок на порядок.

### Анализ требований

Формализация бизнес-процессов заказчика

### Функциональная модель

Иерархия IDEF0-диаграмм

### Проектирование ИС

Архитектура, БД, интерфейсы

### Реализация

Код, тестирование, внедрение

# История и предпосылки

1

## 1973 — SADT

Дуглас Росс (SofTech) разрабатывает методологию **Structured Analysis and Design Technique** — первый формальный язык функционального моделирования для описания сложных систем.

2

## 1981 — Программа ICAM

ВВС США в рамках программы **Integrated Computer-Aided Manufacturing** адаптируют SADT и публикуют IDEF0 как открытый стандарт для промышленного применения.

3

## 1993 — FIPS 183

Национальный институт стандартов США (NIST) утверждает IDEF0 как федеральный стандарт обработки информации.

4

## 1998 — IEEE 1320.1

Международная стандартизация нотации. IDEF0 становится частью семейства методологий структурного анализа наряду с DFD и UML.

---

## Три кита структурного анализа

### IDEF0

**Функциональное моделирование.** Акцент на том, *что* делает система и какие ресурсы задействованы. Статическая модель функций.

### DFD

**Потоки данных.** Акцент на движении данных между процессами и хранилищами. Динамическая модель информационных потоков.

### UML

**Объектно-ориентированное моделирование.** Акцент на структуре объектов, поведении и взаимодействии. Применяется на этапе проектирования ПО.

# Базовые принципы структурного анализа

## 1 Разделяй и властвуй — декомпозиция сложных систем

Любая сложная система разбивается на подсистемы, каждая из которых анализируется независимо. Декомпозиция продолжается до тех пор, пока каждая функция не станет атомарной и понятной без дополнительных пояснений.

## 2 Иерархическое упорядочивание

Результаты декомпозиции организуются в строгую иерархию: контекстная диаграмма → диаграммы первого уровня → диаграммы второго уровня и т.д. Каждый уровень добавляет детализацию, не нарушая целостности верхнего уровня.

## 3 Абстрагирование

На каждом уровне модели отображается только существенная информация, релевантная данному уровню детализации. Несущественные детали скрываются — это позволяет управлять сложностью и фокусировать внимание аналитика.

## 4 Формализация и непротиворечивость

Модель должна быть однозначной: каждый элемент имеет строго определённую семантику. Стрелки, пересекающие границы диаграмм, должны быть согласованы между уровнями — это обеспечивает целостность всей модели.

Эти четыре принципа применимы не только к IDEF0, но и к любому методу структурного анализа — они составляют философию инженерного мышления.

# Основные компоненты IDEFo

## Функциональный блок

Прямоугольник, обозначающий **действие, процесс или преобразование**. Именуется глаголом или глагольным оборотом (например: «Обработать заявку», «Сформировать отчёт»).

Каждый блок — это «чёрный ящик», который принимает входы и производит выходы согласно управляющим воздействиям при наличии необходимых механизмов.

## Стрелки (потоки)

Направленные линии, представляющие **данные или объекты**, связывающие функциональные блоки между собой и с внешней средой. Стрелка — это не поток управления (как в блок-схеме), а поток *данных или ресурсов*.

## Модель ICOM — четыре обязательных типа стрелок

### Input (Вход)

Левая сторона блока. Данные, преобразуемые функцией.

### Control (Управление)

Верхняя сторона. Условия и правила выполнения.

### Mechanism (Механизм)

Нижняя сторона. Ресурсы, исполнители, инструменты.

### Output (Выход)

Правая сторона. Результат выполнения функции.

**ICOM** — аббревиатура: **I**nput, **C**ontrol, **O**utput, **M**echanism. Все четыре типа обязательны для полноты модели.

# Семантика стрелок IDEF0

Правильная классификация стрелок — ключевой навык при построении IDEF0-модели. Ошибка в типе стрелки искажает смысл всей функции.

| Сторона блока | Тип стрелки | Значение | Пример для программиста |
|---|---|---|---|
| Левая | **Вход (Input)** | Что преобразуется функцией | Исходные данные, запросы пользователя, необработанные записи |
| Верхняя | **Управление (Control)** | Как и когда выполнять функцию | Алгоритмы, бизнес-правила, требования, стандарты, регламенты |
| Нижняя | **Механизм (Mechanism)** | Кто или чем выполняется функция | Исполнители, серверы, СУБД, программные инструменты, API |
| Правая | **Выход (Output)** | Результат выполнения функции | Отчёты, обработанные данные, записи в БД, уведомления |

## Типичные ошибки классификации

### Вход vs. Управление

Данные, которые *преобразуются* — это Вход. Данные, которые *управляют* процессом, но не изменяются — это Управление. Пример: «Список книг» — Вход; «Правила каталогизации» — Управление.

### Механизм vs. Вход

Ресурс, который *используется*, но не потребляется — это Механизм. Ресурс, который *потребляется* в процессе — это Вход. Пример: «Библиотекарь» — Механизм; «Бланк заявки» — Вход.

### Выход vs. Механизм

Выход — это то, что *создаётся* функцией. Механизм — то, что *обеспечивает* выполнение. Пример: «Сформированный отчёт» — Выход; «Генератор отчётов» — Механизм.

# Модель IDEF0 как иерархия диаграмм

## Уровни иерархии

01

### Контекстная диаграмма (A-0)

Вершина иерархии. Один функциональный блок, представляющий всю систему целиком. Определяет границы модели, внешние входы, выходы, управление и механизмы.

02

### Диаграммы декомпозиции (A1, A2...)

Детализация каждого блока контекстной диаграммы. Каждый блок раскрывается в отдельную диаграмму с 3–6 подфункциями. Стрелки родительского блока становятся граничными стрелками дочерней диаграммы.

03

### Диаграммы нижнего уровня

Декомпозиция продолжается до атомарных функций, которые не требуют дальнейшего разбиения. Глубина иерархии определяется сложностью системы.

## Дерево узлов

Дерево узлов — вспомогательная диаграмма, отображающая иерархическую структуру всей модели в виде дерева. Используется для навигации по модели и контроля полноты декомпозиции.

Нумерация узлов: **A-0** (контекст) → **A0** (первый уровень) → **A1, A2, A3...** (блоки первого уровня) → **A11, A12...** (блоки второго уровня).

## FEO-диаграммы (For Exposition Only)

Упрощённые диаграммы, создаваемые исключительно для иллюстрации отдельных аспектов модели. **Не являются частью официальной модели** и не подчиняются всем правилам IDEF0. Используются в презентациях и документации для заказчика.

Ключевое правило: стрелки, входящие в блок на родительской диаграмме, должны появляться как граничные стрелки на дочерней диаграмме — это обеспечивает **согласованность иерархии**.

# Правило 3–6 блоков и принципы построения диаграмм

## Правило 3–6 блоков на диаграмме

**Минимум 3, максимум 6** функциональных блоков на одной диаграмме декомпозиции. Это ограничение обусловлено когнитивными возможностями человека: более 6 объектов одновременно воспринимаются с трудом.

- Менее 3 блоков — декомпозиция нецелесообразна, функцию можно описать текстом
- Более 6 блоков — диаграмма перегружена, необходима дополнительная группировка
- Оптимум: 4–5 блоков на диаграмму

## Принцип доминирования — ступенчатое расположение

Блоки на диаграмме располагаются по диагонали: наиболее доминирующая (главная) функция — в левом верхнем углу, наименее доминирующая — в правом нижнем. Это отражает логический порядок выполнения и визуально подчёркивает иерархию важности.

## Нумерация блоков и связность

Каждый блок получает уникальный номер в рамках диаграммы (1, 2, 3...). Полный адрес блока формируется как номер родительского узла + номер блока: блок 2 на диаграмме A1 получает адрес **A12**.

Связность диаграмм обеспечивается через **туннельные стрелки** (в скобках) — стрелки, которые не передаются на родительский или дочерний уровень намеренно.

---

**3–6**

Блоков на диаграмму

Когнитивный оптимум восприятия

**1**

Блок в A-0

Контекстная диаграмма всегда содержит ровно один блок

**∞**

Уровней иерархии

Глубина декомпозиции не ограничена стандартом

**4**

Типа стрелок

ICOM — обязательная модель для каждого блока

# Контекстная диаграмма: ключевые определения

Перед построением контекстной диаграммы аналитик обязан формально определить четыре параметра модели. Без этого шага модель теряет однозначность и практическую ценность.

1

## Субъект моделирования

**Что моделируем?** Чёткое определение системы или процесса, который является объектом анализа. Формулируется как существительное: «Система учёта книг», «Процесс обработки заявок».

2

## Цель моделирования

**Почему и что должна показывать модель?** Цель определяет, какие аспекты системы будут отражены, а какие — намеренно опущены. Пример: «Показать информационные потоки между отделами для выявления узких мест».

3

## Точка зрения

**Чья позиция?** Модель строится с позиции конкретного участника или роли: библиотекаря, системного администратора, директора. Одна и та же система с разных точек зрения даёт разные модели.

4

## Границы модели

**Что внутри, что снаружи?** Явное определение того, что входит в область моделирования, а что является внешней средой. Граница фиксируется стрелками контекстной диаграммы.

Эти четыре параметра фиксируются в **заголовке модели** (Model Header) и сопровождают каждую диаграмму. Изменение любого из них означает создание новой модели.

# Практический пример: Учёт книг в библиотеке

## Постановка задачи

**«Разработка модуля учёта книг в библиотеке»**

## Шаг 1 — Формулирование параметров модели

| Параметр | Значение |
|---|---|
| Субъект | Процесс учёта книжного фонда библиотеки |
| Цель | Выявить информационные потоки для проектирования модуля ИС |
| Точка зрения | Библиотекарь (оператор системы) |
| Границы | Внутри: приём, выдача, возврат, списание книг. Снаружи: поставщики, читатели |

## Шаг 2 — Контекстная диаграмма A-0

### Вход (Input) — левая сторона

Новые книги от поставщиков · Заявки читателей на выдачу · Возвращённые книги

### Управление (Control) — верхняя сторона

Правила каталогизации · Регламент выдачи и возврата · Нормативы списания

### Механизм (Mechanism) — нижняя сторона

Библиотекарь · Программный модуль учёта · СУБД каталога

### Выход (Output) — правая сторона

Обновлённый каталог · Карточки выдачи · Акты списания · Отчёты о фонде

## Шаг 3 — Построение диаграммы

На основе сформулированных параметров строится контекстная диаграмма A-0 с единственным блоком **«Вести учёт книжного фонда»**. Далее выполняется декомпозиция на диаграмму A0 с 4–5 подфункциями: приём книг, каталогизация, выдача, возврат, списание.

Задание: самостоятельно выполнить декомпозицию блока «Вести учёт книжного фонда» до уровня A0, соблюдая правило 3–6 блоков и принцип доминирования. Результат оформить в любом CASE-инструменте (AllFusion BPwin, Ramus, draw.io).